Trabajo Práctico - TP2

[7451/9515] Algoritmos y Programación II

Primer cuatrimestre de 2022

Alumno: Albornoz Tomé, Rodolfo Valentin
Padron: 107975
Email: ralbornozt@fi.uba.ar

Índice

1. Introducción **2**

2. Juego **2**
2.1. Cómo jugar 2
2.2. Interfaz y lógica 2

3. Implementación del TP **2**
3.1. Historial, dificultades y justificacion 2
3.2. Estructura de la sala 3
3.3. Definicion de TDAs a usar 3
3.4. Cambios en el TP1 3
3.5. Implementacion de las nuevas funciones 4

4. Pruebas **4**
4.1. Pruebas originales que fallaban 4
4.2. Pruebas agregadas 5

5. Diagramas **5**

1. Introducción

El objetivo de este TP fue **completar** el juego **implementado en el TP1**, ahora incorporando los **TDAs vistos en la cursada** (Lista, ABB, Hash, etc)

2. Juego

2.1. Cómo jugar

Provisto con el **makefile**, por consola se debe escribir **make escape**, y luego escribir **./nombre_ejecutable archivo_objetos archivo_interacciones**

De esta forma, si los archivos son correctos, se **inicia el juego**, y todas las **ayudas e instrucciones** los provee el mismo.

2.2. Interfaz y lógica

El juego tiene la mayor cantidad de **ayudas y tips posibles**, mostrando mensajes en el caso de ingresar un **comando incorrecto** (Ya sea por escribir **una mayuscula, minuscula o un espacio de mas**), en el caso de necesitar **ayuda para un comando**, etc.

Algunas caracteristicas de su lógica e interfaz:

- Se valida la **cantidad de argumentos recibidos**.
- Se imprime una **pantalla de inicio**.
- El juego cuenta con un **contador** que dice **cuántos comandos/intentos** fueron necesarios para **escapar de la sala**, y se muestra al escapar de la misma.
- Existe un comando para imprimir la **informacion conocida** (Objetos poseidos y conocidos) del juego.
- Los comandos válidos del juego tienen entre **1 a 3 palabras**. El **procesamiento de comandos** (Decidir si es válido, y si lo es, **ejecutar la accion correspondiente**) se realiza según la **cantidad de palabras** (Sin contabilizar y tomando como **vacias** las palabras que no se insertaron para llegar a la cantidad de 3).
- **Siempre se ejecuta una interacción si es posible hacerlo**.
- La **ayuda** contiene la **explicacion de cada comando**, unos tips sobre los objetos **conocidos y poseidos**, y una explicacion del **formato de las interacciones**.
- Existen **validaciones** para tomar como correctos comandos que **tienen primera letra mayuscula o minuscula**, o que tienen un **espacio antes o después de cierta palabra**.
- Si la cantidad de palabras **no es correcta** o la **interaccion enviada es inválida**, se escribe una ayuda.
- El **formato de las interacciones** es: **verbo primer objeto segundo objeto**. En caso de ser una interaccion de 1 objeto, se omite el segundo objeto.

3. Implementación del TP

3.1. Historial, dificultades y justificacion

Empecé definiendo la **interfaz de usuario** (Mensajes a mostrar), mostrando los **mensajes básicos** que se le muestran al usuario (Comandos, ayuda, etc).

Definí el **alcance** del juego, especificando que termina cuando **se presiona el comando salir** o cuando se **escapa de la sala**, para lo cual implementé primero el **sala_escape_exitoso**, devolviendo un **booleano definido dentro de la estructura de la sala** que indica si se escapó o no.

Lo próximo que hice fue **mejorar y simplificar mi implementación del TP1** e **incorporarle a las funciones implementadas el uso de los TDAs**

Segui con la implementación de **sala_describir_objeto**, **sala_agarrar_objeto** y las de **obtener_nombres** (Las cuales eran todas casi iguales) y no tuve muchos problemas al principio.

Cuando segui con la de **ejecutar_interacion** tuve muchos problemas por confundirme con las condiciones de **cuando se podia o no realizar cada accion**. Esto también me llevó problemas por conflictos con el **agarrar** y **describir** (Por ejemplo, errores de memoria al intentar describir un objeto no existente, asi que esto me atrasó un buen tiempo.

Una vez arreglado todo lo de la implementación, **terminé las pruebas y la parte de la lógica del juego y la interfaz**.

3.2. Estructura de la sala

Para la sala, decidí que se almacenara dentro:

- Un **booleano** para saber cuando se pudo **escapar de la sala**.
- 3 punteros a **hash**, cada uno conteniendo información de los **objetos de la sala** (Uno con **todos los objetos**, otro con los **conocidos**, y otro con los **poseidos**).
- Un puntero a **lista** que contiene dentro la información de las **interacciones posibles** dentro de la sala.

3.3. Definicion de TDAs a usar

Elegí almacenar los **objetos** (**Totales**, **conocidos** y **poseidos**) en un **hash**, porque como los nombres de los objetos **no se repetian**, y además el hash **puede realizar una busqueda rápida**, me servía para cuando necesitara acceder a los objetos de la sala. Lo hice con el criterio de que los **nombres de los objetos** eran las **claves**, y **los punteros a estructuras objeto** eran los **valores**.

Por el lado de las **interacciones**, elegí una **lista** porque la mayoría de **campos del archivo** tenian valores que se repetian (Excepto la **descripcion**, pero no lo veía útil como **clave de un hash**), asi que no me pareció conveniente usar **hash** o **abb**. Abb tampoco me pareció util usar porque **podrían repetirse los elementos en el arbol** y eso me causaría problemas, además de que **no necesitaba guardar las interacciones en un orden como en el de un abb** (Con 'mayores' por un lado y menores por otro). Asi que por descarte utilicé una **lista**, donde cada nodo almacena un **puntero a una estructura de interacción**.

3.4. Cambios en el TP1

- En **sala_crear_desde_archivos**, **inicialicé todos los campos** y los **TDAs utilizados** de la sala.
- En las funciones de **guardado de objetos** (leer_objetos, guardar_objeto), modularicé todo, y guardé los objetos dentro de un **hash**. Tuve que modificar la función porque **no me era posible pasarle un puntero a la cantidad de objetos para modificarla**. Además incorporé el **primer objeto leido** dentro del **hash de objetos conocidos**.
- En las funciones de **guardado de interacciones** (leer_interacciones, guardar_interaccion), modularicé todo y guardé las interacciones dentro de una **lista**. Tuve que modificar la función porque **no me era posible pasarle un puntero a la cantidad de interacciones para modificarla**.
- En **sala_obtener_nombre_objetos**, cree una **estructura** (nombres) que guardara el **vector de nombres creado** y la **cantidad de nombres** (Inicialmente 0), y usando la funcion **hash_con_cada_clave** (Para poder acceder a cada clave) junto con la función adecuada (guardar_nombre), **guardo los nombres de cada objeto (Claves) en la sala dentro del vector de nombres en la estructura**, y **actualizo la cantidad de nombres**.
- En **sala_es_interaccion_valida**, creo una estructura **elementos interaccion** que guarda los **objetos de la inteccion, el verbo, y si la interaccion es valida**. A partir de ella, y usando el **iterador interno** de la lista, **comparo con todas las interacciones de la lista**, a ver si los objetos y verbo de alguna **coincide con ella**. Si alguna coincide, se **deja de recorrer** en la lista.

- En **sala_destruir**, se llaman a las **funciones de destruccion correspondientes para lista y hash**, destruyendo los **hash de objetos** y la **lista de interacciones**. Para el **hash de objetos totales** y la **lista de interacciones**, se envia un destructor (free) para liberar de la memoria **cada elemento de las estructuras**. Para los otros 2 hash, simplemente se **destruyen las estructuras**.

3.5. Implementacion de las nuevas funciones

- **OBTENER NOMBRES DE OBJETOS**: Tanto para los **objetos de la sala** como los **conocidos** y los **poseidos**, se realizan acciones muy similares (Por eso todas usan la **misma funcion**).

  Se reserva un vector del tamaño de la **cantidad de objetos del tipo correspondiente**. Si la **cantidad es 0**, se retorna un **vector vacio de cantidad 0** (No -1). Después se utiliza la **estructura de nombres** junto al **iterador interno del hash** para guardar en **cada posicion del vector** los nombres de los **objetos del tipo correspondiente** y **actualizar la cantidad en la estructura**. Al final se retorna el vector.

- **AGARRAR OBJETO**: **NO se agarra el objeto** si se **cumple alguno** de los siguientes casos:

  -**Ya poseo** el objeto.

  -**No conozco** el objeto.

  -El objeto **no es asible**.

  Si no se cumple ni uno, entonces se **incorpora el objeto en el hash de poseidos**, y se lo **elimina del hash de conocidos**.

- **DESCRIBIR OBJETO**: Se trae la descripcion del objeto **solo si lo poseo O lo conozco**.

- **EJECUTAR INTERACCION**: Se utiliza el **iterador externo de la lista**, tomando **cada interaccion de la misma**, comparando si sus objetos y verbo coinciden con los **enviados a la funcion**, y si es el caso, se llama a una función que **evalua el tipo de accion de la interaccion** y hace algo en base a eso, sumando 1 a la **cantidad de ejecuciones** si es **válida para realizar** y **mostrando el mensaje asociado a la interaccion** si es que **hay alguno** y si la funcion de mostrar mensaje **no es nula**.

  - Si se realiza una accion del tipo **mostrar mensaje**, se muestra el mensaje solo si: **Conozco O poseo** el primer objeto, y **conozco el segundo**.
  - Si se realiza una accion del tipo **descubrir objeto**, puedo descubrir el objeto solo si **conozco O poseo el primer objeto**, **Conozco el segundo objeto**, **NO conozco NI poseo el objeto de la accion** y el **objeto de la accion NO fue eliminado de los objetos de la sala**. Si puedo descubrir, entonces **inserto en conocidos el objeto de la accion**.
  - Si se realiza una accion del tipo **reemplazar objeto**, se reemplaza el objeto si poseo el primero y **conozco el segundo**. Si esto se cumple, después no puedo reemplazar si **el objeto de la accion fue eliminado de la sala**.

    Luego, inserto el **objeto de la accion en conocidos**, **quito de conocidos el segundo objeto** y **quito de los objetos de la sala el segundo objeto**
  - Si se realiza una accion del tipo **eliminar objeto**, se elimina el objeto en los siguientes casos: **Conozco el objeto** (Y lo quito de conocidos), **poseo el objeto** (Y lo quito de poseidos). Si **al menos lo poseo o al menos lo conozco** y **aun no lo eliminé**, lo elimino. En cualquier otro caso, **NO puedo eliminarlo** (Por ejemplo, **no lo conozco ni lo poseo** pero **aun está en los objetos de la sala**)

4. Pruebas

4.1. Pruebas originales que fallaban

- **"Se leyeron 9 objetos"** y **"Se leyeron 9 interacciones"** (Errores de compilacion).

  Motivo: Ahora la **estructura de la sala** está en sala.c y no en sala.h.

Solucion: Definí un archivo 'auxiliar.h', que dentro contiene 2 funciones que **reciben la sala** y devuelven la **cantidad de objetos** e **interacciones**. Dentro de auxiliar.h tuve que **incluir la estructura de la sala**, porque la cantidad de objetos es **interna del hash** y la cantidad de interacciones es **interna a la lista**, y estas 2 estructuras están **dentro de la sala** asi que no se podia acceder a ellas.

- **"Todos los nombres de objeto son los esperados"**

  Motivo: Como los objetos ahora se guardan en un **hash**, al buscarlos en la **tabla de hash**, no se encontrarán en el **mismo orden en que fueron insertados**.

  Solucion: Al igual que la prueba de la cantidad de objetos e interacciones, **no me era posible acceder a la estructura de objetos para usar una primitiva de hash**, por lo que usé una función de 'auxiliar.h' (sala_contiene_objeto) que a partir de la **sala** y el **nombre del objeto**, podía traer si la sala **contenia un objeto con ese nombre**. Se invocaba a esta función con **cada nombre del vector de esperados**.

4.2. Pruebas agregadas

Entre las **nuevas pruebas añadidas** al archivo pruebas.c, se encuentran:

- Pruebas con **cosas NULL** (Sala, nombres, etc)
- Pruebas de **comportamiento esperado**, ya sea **correcto** e **incorrecto** (Por ejemplo, los casos posibles en que se **puede agarrar o no un objeto**)
- Mayormente pruebas con el **ejecutar_interaccion**, realizando acciones de **todos los tipos posibles** y en sus casos **correctos** o en casos **extremos**.
- Las únicas pruebas que se podrían considerar **de caja blanca** son las **arregladas del tp1**.

5. Diagramas

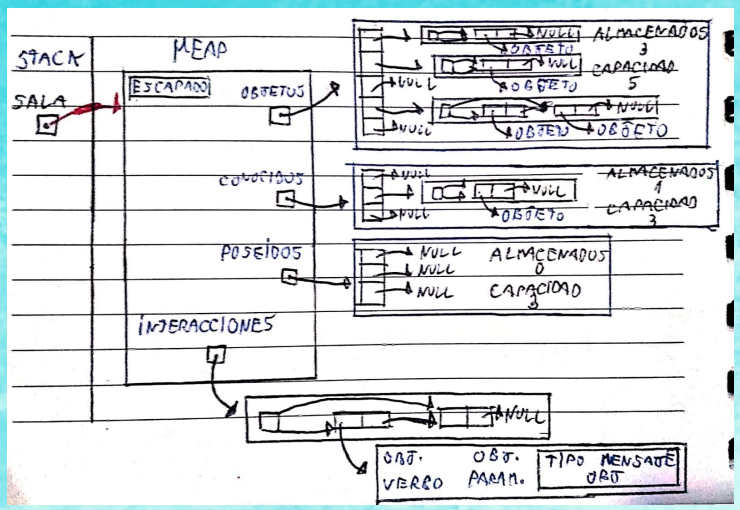

Figura 1: Nueva estructura de la sala, incorporando los TDAs Lista y Hash. Así comienza la sala inicialmente, conociendo el primer objeto